

7-Day AI Dev Curriculum for a Python + Kubernetes Engineer

This plan is designed for someone with:

- Python experience
- Kubernetes familiarity
- 2 hours/day available
- preference for focused learning + hands-on practice

Goal: reach a practical working level in RAG + LangGraph in 7 days.

Overview

Week focus:

1. Learn how LLMs are called and how prompts + tools work
2. Build a small retrieval pipeline
3. Learn LangGraph fundamentals
4. Add tool use and agent loops
5. Build a useful project around your domain knowledge

Daily time table (2 hours/day)

Day 1 — LLM basics and raw API calls

Time:

- 20 min: set up venv and project structure
- 30 min: understand prompt, system message, user message
- 30 min: call an LLM via SDK
- 20 min: inspect response structure
- 20 min: try a tool call manually

Key concepts:

- model, messages, system prompt, temperature, max tokens
- function/tool calling basics

- what the LLM actually sees

Hands-on:

- Write ``basic_llm.py``
- Send a simple prompt
- Send a tool call request
- Print and inspect response JSON

Deliverable:

- 1 script that makes a successful LLM call

Day 2 — RAG from scratch

Time:

- 20 min: understand embeddings and vector search
- 30 min: create sample docs
- 30 min: chunk and embed text
- 20 min: store in Chroma or FAISS
- 20 min: retrieve and inject context into prompt

Key concepts:

- embeddings
- chunking
- vector stores
- top-k retrieval
- context injection

Hands-on:

- Create 5–10 Markdown/text docs
- Chunk them into small pieces
- Embed them
- Query with a question
- Inject the top results into the model prompt

Deliverable:

- ``rag_basic.py`` that answers a question using retrieved docs

Day 3 — LangGraph basics

Time:

- 20 min: learn nodes, edges, state
- 30 min: build a basic graph
- 30 min: implement router logic
- 20 min: pass state between nodes
- 20 min: test the graph

Key concepts:

- `StateGraph`
- nodes
- edges
- conditional routing
- state updates

Hands-on:

- Build a graph:
 - user question -> decide whether retrieval is needed -> retrieve -> answer
- Test for both general and document-specific questions

Deliverable:

- a minimal LangGraph workflow that returns an answer

Day 4 — Tool-calling agent in LangGraph

Time:

- 20 min: create 2–3 simple tools
- 30 min: wire tools into graph
- 30 min: model chooses when to call a tool
- 20 min: loop until final answer
- 20 min: test with edge cases

Key concepts:

- tool calling
- agent loop
- stateful execution
- deciding when to stop

Hands-on:

- Add tools like:

- doc search - calculator - current time

- Allow the agent to call tools and continue reasoning

Deliverable:

- a tool-using LangGraph agent

Day 5 — Build a domain-specific project

Time:

- 20 min: decide project scope
- 30 min: gather domain docs
- 30 min: build RAG index
- 20 min: integrate LangGraph workflow
- 20 min: test 5–10 sample questions

Best project choices:

- Kubernetes docs assistant
- Platform troubleshooting assistant
- Internal operations Q&A; bot
- GitHub repo understanding assistant

Hands-on:

- Use your Kubernetes knowledge as the domain
- Query with examples like:

- How does a Deployment differ from a StatefulSet? - What does a Kubernetes readiness probe do? - How do I debug a crashlooping pod?

Deliverable:

- a small domain-focused RAG + agent project

Day 6 — Quality, reliability, and prompt tuning

Time:

- 20 min: evaluate 10 questions
- 30 min: tune chunk size and retrieval depth
- 30 min: improve prompt wording

- 20 min: add fallback path if retrieval is empty
- 20 min: test edge cases

Key concepts:

- retrieval quality
- chunk strategy
- prompt clarity
- fallback behavior

Hands-on:

- Try different chunk sizes
- Compare top-k values
- Add “not enough information” behavior
- Make the result more grounded

Deliverable:

- improved project that answers better and doesn't overreach

Day 7 — Final project, README, and demo

Time:

- 30 min: clean project structure
- 30 min: write README with setup steps
- 30 min: do final testing
- 20 min: write a 3-minute walkthrough
- 10 min: save a sample answer

Key concepts:

- packaging and documentation
- clear project story
- demo flow

Hands-on:

- Structure code as:

- `app/` - `data/` - `tools/` - `graphs/` - `README.md`

- Record a short demo using 3 questions

Deliverable:

- ready-to-demo project + presentation notes

Suggested project stack

- Python
- OpenAI or Anthropic SDK
- LangGraph
- Chroma or FAISS
- LangChain or plain Python + vector store
- Markdown or text documents as data source

Suggested folder structure

```
project/
├── app/
│   ├── main.py
│   ├── rag.py
│   ├── graph.py
│   └── tools.py
├── data/
│   ├── docs/
│   └── sample_questions.txt
├── README.md
├── requirements.txt
└── .env
```

Success checklist by end of week

By Day 7, you should be able to say:

- I can call an LLM from Python
- I can build a basic RAG pipeline
- I understand embeddings + retrieval
- I can build a LangGraph workflow
- I can add tools to an agent
- I can explain my project clearly

Recommended order of learning

Do not begin with heavy framework abstractions. Learn in this order:

1. raw LLM API call
2. RAG basics
3. LangGraph fundamentals
4. tool-calling loop
5. your project

This order makes the framework easier to understand.

What not to worry about yet

Do not spend too much time on:

- multi-agent orchestration
- production-scale memory systems
- prompt optimization at extreme depth
- fancy frontends
- huge eval suites

The priority is a working prototype and strong understanding of the flow.

Minimal goal for this week

If you do only the essentials, your goal is:

- build one working RAG app
- build one LangGraph agent
- answer 5–10 useful questions on your chosen topic
- present it confidently

That is an excellent first milestone.